

RMS Dynamic Layout 최적화 모형 개선

연구 진행 보고: 기준 논문 재현, reformulation, 확장(relocation / shared resource)

스마트생산시스템 연구실

2026년 7월

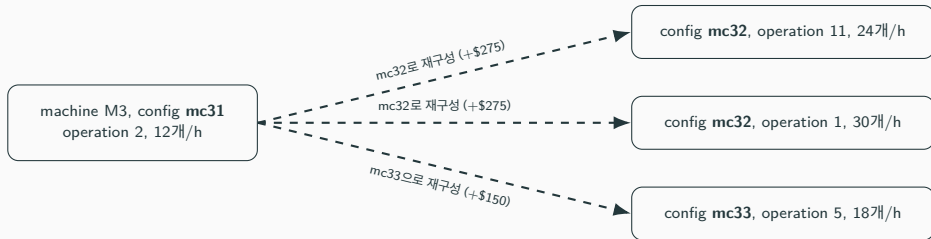
1. **문제:** 기간별로 수요가 변하는 환경에서 RMT의 구매·설치 위치·재구성·자재 흐름을 동시에 결정하는 dynamic layout 문제. 기존 논문(Saffar et al., IJPR 2025)이 MILP로 정식화.
2. **동기:** 기존 논문의 MILP는 위치 16개 예제에 3시간 이상 소요. 원인을 진단하고 formulation을 개선하는 것이 목표.
3. **수행 내용:** LP bound 진단, reformulation 3종 구현·검증, 확장 2종(relocation, shared resource), 규모 확장 실험.
4. **주요 결과:** network reformulation이 동일 최적해를 4.5배 빠르게 도출하며, 문제 규모가 클수록 격차가 커짐. 부수적으로 기존 논문의 reported 최적값 2건이 실제 최적이지 않음을 확인.

1. 배경 및 문제 정의

Reconfigurable Manufacturing System: 수요 변화에 대응해 설비 구성을 변경할 수 있는 생산 시스템.

RMT (Reconfigurable Machine Tool): RMS의 핵심 설비.

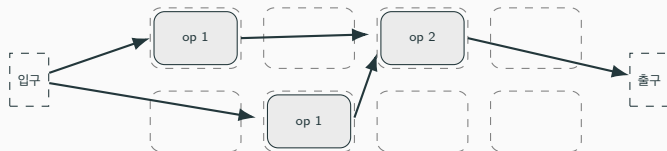
- basic module(고정) + auxiliary module(교체 가능)로 구성
- auxiliary module 추가/제거 시, 수행 가능한 operation과 생산 속도가 달라짐
- 재구성 비용 = 추가 module 수 × 단가 + 제거 module 수 × 단가



하나의 상태에서도 (대상 configuration) × (수행 operation) 조합만큼 재구성 옵션이 존재 — 이 분기가 위치·기간마다 반복되고, “구매 vs 재구성” 트레이드오프와 결합된다.

문제 정의: dynamic layout design

설비를 설치할 수 있는 후보 위치들과 자재가 출입하는 입구/출구가 주어지며, 계획 기간동안의 수요가 주어짐.
아래는 한 기간의 layout snapshot:



점선 = 빈 위치
회색 상자 = 설치된 RMT
화살표 = 자재 흐름

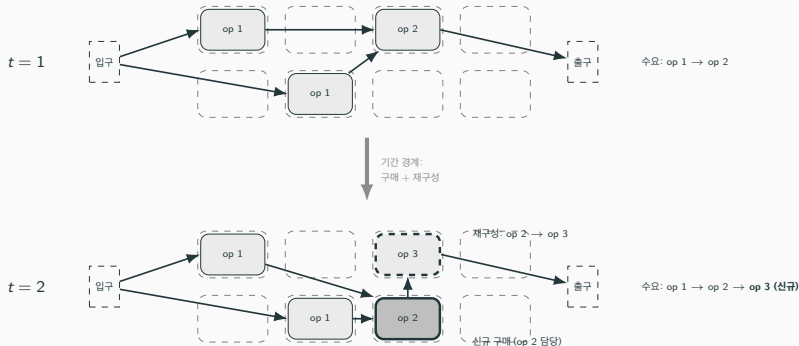
결정변수 (전 기간 동시 최적화):

1. 구매: 어떤 configuration의 RMT를 어느 위치에 설치할 것인가
2. 재구성: 기간 경계에서 어떤 RMT의 configuration을 변경할 것인가
3. 자재 흐름: 각 기간에 어느 기계에서 어느 기계로 얼마나 운반할 것인가

목적함수 = 구매 비용 + 재구성 비용 + 운반 비용(운반량 $\times$ Manhattan 거리). 세 결정이 상호 의존적이므로
통합 최적화가 필요하다.

문제 정의: 시간이 바뀌면 layout 결정이 반복된다

기간마다 수요(operation 구성·양)가 달라지므로, 기간 경계에서 **신규 구매**와 **재구성**이 발생하고 자재 흐름도 다시 설계된다. 예: $t = 2$ 에 operation 3의 수요가 새로 등장하는 경우 —



동일한 위치 grid이지만 시간에 따라 설치 기계·configuration·자재 흐름이 모두 달라진다.

예시: 구매 vs 재구성 트레이드오프

기간 3부터 operation 11(configuration mc32 필요)의 수요가 발생하는 경우:



대안 1: 기간 3에 mc32 직접 구매

비용 = \$1,825

대안 2: 기간 1에 mc31 구매(\$780) 후

기간 3에 재구성(+\$275) = **\$1,055**

이러한 트레이드오프가 위치 선택·운반 비용과 결합되어 다수 동시에 발생하므로 최적화 모형이 필요하다. (위 사례는 기준 논문 Example 2에서 실제로 최적해에 나타남.)

Saffar, Moghaddam & Huang (2025), *IJPR* 63(17): 이 문제를 MILP로 정식화. 본 연구는 이 논문의 모형을 기준(base)으로 삼는다.

논문이 보고한 계산 시간 (GAMS):

- Example 1 (위치 16, 4기간, 부품 1종): 11,254초
- Example 2 (위치 20, 부품 3종): 25,887초

자체 재현 (Gurobi): Example 2는 36,000초 후에도 optimality gap 0.88% 잔존.

연구 질문

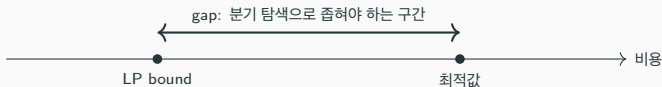
소규모 예제에서 이 정도면 실규모 적용 불가.

1. 계산이 느린 원인은 무엇인가
2. formulation을 바꾸면 개선되는가

2. 진단: base model의 LP bound

MILP는 branch-and-bound로 풀린다:

1. 정수 조건을 제거한 **LP relaxation**을 풀어 최적값의 하한(**LP bound**)을 얻음
2. incumbent(현재 최선 해)와 bound의 차이(**gap**)가 0이 될 때까지 분기 탐색



전제가 되는 원리

LP bound가 최적값에 가까울수록(formulation이 강할수록) 탐색량이 줄어 빨리 풀린다. 동일한 문제라도 정식화 방식에 따라 LP bound가 달라진다 — 이것이 **reformulation** 연구의 근거다.

진단: base model의 LP relaxation이 약한 이유

LP relaxation에는 정수 조건이 없으므로, base model에서는 다음 해가 허용된다:

원인 1: fractional 구매

수요 80을 처리속도 24인 기계로 처리하려면 정수해는 4대 필요. LP는 $80/24 = 3.33$ 대만 구매.

원인 2: 재구성 비용 과소계상

상태전이 제약 (식 (5)–(7))이 부등식(implication) 이어서, 상태를 여러 configuration에 fractional하게 분산시키면 재구성 비용을 일부만 지불 가능.

측정 결과 (Example 1, 최적값 22,910 기준):

비용 항목	LP relaxation	정수 최적해	진단
구매	10,019	$\approx 11,025$	fractional 구매 (원인 1)
재구성	841	1,925	56% 과소계상 (원인 2)
운반	10,065	10,132	거의 tight — 문제 없음
LP bound	20,925 (gap 8.7%); solver의 general cut 적용 후에도 3.4% 잔존		

3. 문제의 구조: machine layer와 flow layer

base model의 변수를 역할별로 나누면 두 그룹이 된다

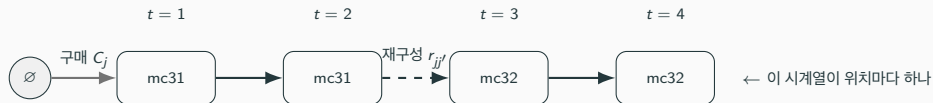
base model의 결정변수 5종을 분류하면, 성격이 다른 두 그룹이 드러난다:

그룹	변수	의미	타입
machine layer	x_{pjl}	위치 p 에 configuration j 의 RMT 구매	binary
	s_{pjlt}	기간 t 에 위치 p 의 상태가 (j, l)	binary
	$y_{pj'jlt}$	기간 t 경계에서 $j' \rightarrow j$ 재구성	binary
flow layer	v_{plt}	기간 t 에 위치 p 의 operation l 처리량	연속
	$f_{plql' t}$	기간 t 에 위치 $p \rightarrow q$ 운반량	연속

- **machine layer** = 설비에 관한 결정 전부: 어떤 기계가, 어느 위치에, 어느 기간에, 어떤 configuration으로 존재하는가. 발생 비용: 구매 + 재구성.
- **flow layer** = 자재에 관한 결정 전부: 각 기간에 자재가 어떤 기계들을 거쳐 얼마나 흐르는가. 발생 비용: 운반.

machine layer: 위치별 기계 상태의 시계열

위치 p 하나를 고정하고 machine layer 변수만 보면, 결정되는 것은 **그 위치의 상태 시계열** 하나다: 설치하지 않거나, $t=1$ 에 구매한 뒤 매 기간 configuration을 유지 또는 재구성.

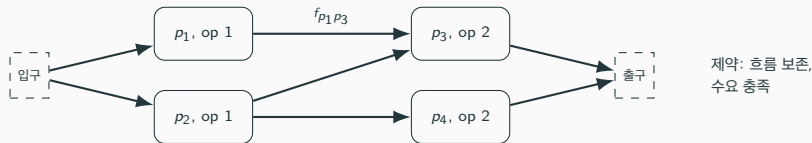


machine layer의 성질:

- 구매·재구성 비용은 위치별 시계열 안에서만 발생 — **위치 간 상호작용이 없다** (자재 운반은 flow layer 소관)
- 변수가 전부 binary $\Rightarrow$ 문제의 조합적 어려움은 전부 이 layer에 있다
- base model은 이 시계열의 논리(유지/재구성으로만 상태 변경 가능)를 부등식 (5)–(7)로 표현한다 — **진단에서 본 원인 2가 발생하는 지점**

flow layer: 기간별 자재 흐름

기간 t 를 고정하고 flow layer 변수만 보면: 그 기간에 가동 중인 기계들을 노드로, 입구 $\rightarrow$ (operation 순서대로) $\rightarrow$ 출구의 흐름을 결정하는 문제다.



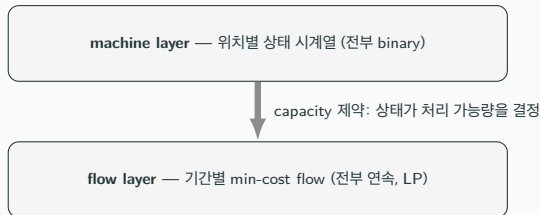
flow layer의 성질:

- 기간별로 독립 (기간 간 재고를 두지 않으므로 기간을 잇는 flow 변수가 없음)
- 변수가 전부 연속이고 제약이 흐름 보존 · 수요 충족뿐 $\Rightarrow$ min-cost flow LP — 그 자체로는 쉬운 문제
- 진단에서 운반 비용 항이 거의 tight했던 이유가 이것 (LP relaxation이 이 layer는 이미 정확히 포착)

두 layer는 capacity 제약 하나로 연결된다

machine layer와 flow layer를 잇는 유일한 제약 — 위치 p 가 기간 t 에 operation l 을 수행하는 상태일 때만, 처리속도 B_{jl} 한도 내에서 flow 처리 가능:

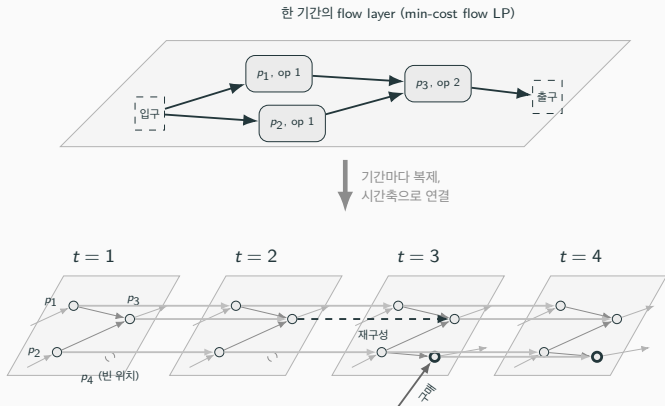
$$v_{plt} \leq \sum_j B_{jl} s_{pjlt} \quad \forall p, l, t$$



이 분해 관점이 주는 결론

flow layer는 LP라 개선할 것이 없고, 진단의 원인 1·2는 모두 **machine layer의 정식화**에서 발생한다. 따라서 machine layer만 다르게 정식화하고 flow layer와 capacity 제약은 그대로 둔다 — 모형 A/B/C가 전부 이 전략이며, 공통 부분이 같아 모형 간 등가성 검증도 용이하다.

전체 구조: flow layer가 기간마다 복제되어 시간축으로 연결된다



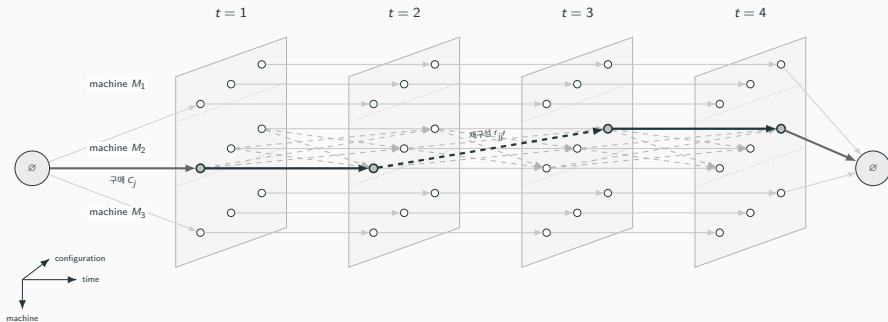
slice 내부 = flow layer (연속, 기간별 독립) · slice 사이 = machine layer (유지 = 실선, 재구성 = 점선, 구매 = 외부 진입)

모형 A/B/C는 모두 slice 내부(flow layer)와 capacity 제약을 공유하고, slice 사이의 machine layer 연결만 다르게 정식화한 것이다.

4. Reformulation

모형 A 아이디어: 상태공간을 시간축으로 전개

한 위치가 가질 수 있는 상태 = (machine, configuration) 조합 전체. 이 상태집합을 $t = 1, \dots, T$ 로 복제하고, 연속 기간의 상태 사이를 arc로 연결한다:

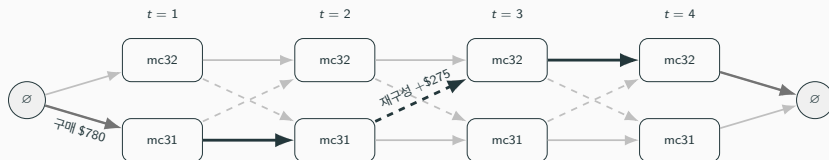


실선 = 유지(비용 0) · 점선 = 재구성 $r_{jj'}$ (동일 machine 내에서만; 그림에는 M_2 만 표시) · $\emptyset$ 진입/이탈 = 구매(C_j)/종료.

위치 하나의 machine lifecycle = source-sink 경로 1개 (미설치 = 경로 없음); 경로의 arc 비용 합 = 구매 + 재구성. 이 그래프가 위치마다 존재하며, capacity 제약으로 flow layer와 연결된다.

모형 A 성질: 왜 LP bound가 개선되는가

단면(machine M_3 의 configuration 2개)만 잘라서 보면:

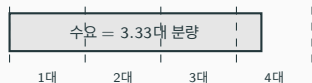


- 노드 점유 변수 $w_{pjl t}$ (base의 s 에 대응) + arc 변수 z (base의 y 와 암묵적 “유지”를 대체)
- 각 노드에 **flow conservation 등식**: 들어온 arc 합 = 노드 점유 = 나가는 arc 합. base의 부등식 (5)–(7)이 등식으로 대체되어, 상태를 fractional하게 분산시켜 재구성 비용을 회피하는 해(진단의 원인 2)가 **LP에서도 불가능해짐**
- 위치 하나의 feasible set이 경로 집합(path polytope)이 되므로 z 는 연속 선언 가능 $\Rightarrow$ binary는 w 만 남아 절반 ($1,856 \rightarrow 832$, Example 1)

B. pattern 기반 (set partitioning)

- pattern = 기계 1대의 전 기간 (configuration, operation) 궤적. 모든 pattern을 사전에 열거하고 비용을 계산한 뒤, 위치마다 최대 1개 선택
- 모형 A 그래프의 source-sink 경로를 통째로 변수화한 것과 동일 = A의 Dantzig-Wolfe reformulation $\Rightarrow$ LP bound가 A와 이론적으로 일치 (실험에서도 전부 일치)
- 한계: pattern 수가 규모에 따라 지수적으로 증가 $\rightarrow$ 대규모에서는 column generation 필요

C. counting cut (base 강화)



- 유효부등식 $\sum s \geq \lceil d_i^t / B_i^{\max} \rceil$ (operation · 기간별 최소 기계 대수)를 base에 추가
- Chvátal-Gomory rounding. 진단의 원인 1(fractional 구매)을 완화
- 구현 비용이 매우 낮아 baseline 강화 수단으로 사용

5. 결과

reformulation은 원 문제와 최적값이 같아야 한다. 모든 실험에 앞서:

- 4개 모형(base / base+cut / network / pattern)이 동일 예제에서 **최적값과 비용 분해(구매/재구성/운반)가 완전히 일치**함을 확인
- 해 추출을 공통 형식으로 통일 — 어느 모형의 해든 동일 CSV·layout 그림으로 출력

부수 발견: 기준 논문의 reported 최적값은 실제 최적이 아님

- Example 1: 논문 보고값 23,082 → 실제 최적 **22,910** (증명 완료). 논문의 기계 배치를 고정하면 23,082가 정확히 재현됨 — 모형 해석의 차이가 아니라 논문 해가 suboptimal
- Example 2: 논문 보고값 36,006 → 100초 내에 **35,791** 발견 (최적성 미증명)

결과 1: 논문 예제 비교

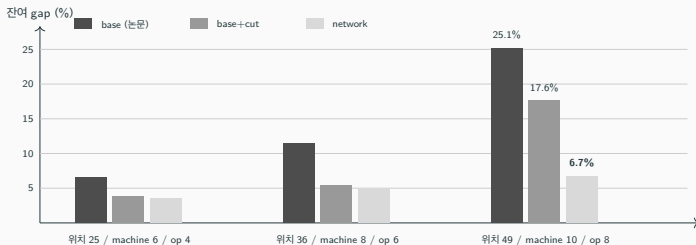
Example 1 (위치 16), solver default 세팅, seed 3개 — 전 run이 최적값 22,910에 도달:

모형	pure LP bound (gap)	solve time (중앙값)	binary 변수
base (논문)	20,925 (8.7%)	73.9초	1,856
base + counting cut	21,401 (6.6%)	30.0초	1,856
network (A)	22,163 (3.3%)	16.4초	832
pattern (B)	22,163 (3.3%)	32.8초	6,944

- network가 base 대비 4.5배 빠름. 특히 network의 *pure* LP bound(22,163)가 base에 Gurobi의 general cut을 모두 적용한 root bound(22,137)보다 높음 $\Rightarrow$ 구조를 반영한 reformulation이 solver의 범용 기법보다 강함
- pattern의 LP bound는 network와 소수점까지 일치 (DW 관계의 실증)
- counting cut은 12개 추가만으로 base를 2.5배 가속

결과 2: 규모 확장 실험

위치 · machine type · operation 수를 키운 합성 인스턴스 3개, time limit 100초. 지표는 100초 후 잔여 optimality gap (낮을수록 우수):



- base: 6.5% → 11.5% → 25.1%로 악화. 최대 인스턴스에서 base의 incumbent는 network의 incumbent보다 9.8% 열등
- network: 3.5% → 4.9% → 6.7% — 규모가 커질수록 reformulation의 효과가 커짐
- pattern: LP bound는 network와 계속 일치하나 변수 수가 231,574개로 열거 한계에 근접 → column generation 도입 근거

6. 후속 실험 (7/12-13): 통합 비교와 valid inequality 강화

§5까지의 상태: network reformulation이 최선, pattern은 CG로 열거 한계 해소. 다음 두 항목을 확인하기 위해 6 단계 실험을 수행했다 (7/12-13):

1. **잔여 LP gap(3% 이상)의 추가 축소** — 단계 1 (counting cut 결합), 단계 3-4 (valid inequality 추가 탐색)
2. **동일 계산 예산에서의 모형별 최종 해 품질** — 단계 2 (통합 비교), 단계 5-6 (강화 모형의 MIP·CG 성능)

실험 기반 정비 (인스턴스·실험 체계):

- 인스턴스를 Data/<family>/<name>으로 재조직 — family = single_part / multi_part, name = paper(논문 예제) / scaled_p25·36·49(합성). **multi-part scaled 3종을 신규 생성** (부품 3~5종, 기간별 phase-in/out으로 part mix가 변하는 구조) → 표준 인스턴스 8개
- 모델 표기 확정: base / base_vi(=base+counting cut) / network / pattern 계열. **pattern은 변형을 항상 명시** — pattern_enum(전체 열거 MIP) vs CG 전략(single / loc_best / mach_best / mach_best_kN). LP bound는 전 변형 동일하나 해 품질은 다르기 때문
- 확장(relocation·shared resource)은 문제 세팅이 달라 별도 실험으로 분리

단계 1: counting cut을 network에 결합 (network_vi)

동기: multi-part scaled에서 base_vi의 LP bound가 network보다 **높은** 역전이 관찰됨 (p25: gap 3.5% vs 6.7%). 부품이 많으면 구매 총 비중이 커지는데, counting cut은 구매 총을, network는 재구성 총을 조인다 — 서로 다른 총이므로 **결합하면 둘 다 지배할** 것으로 예상.

결과 (network_vi = network + counting cut; pure LP gap, best known 대비):

인스턴스	base_vi	network	network_vi	비고
single/paper	6.6%	3.3%	1.3%	최적 증명 1.9초 (base 40초) 역전 해소
single/p49	14.3%	6.4%	3.7%	
multi/p25	3.5%	6.7%	2.8%	
multi/p49	10.9%	11.6%	8.5%	

- 8개 인스턴스 전부에서 base_vi · network 양쪽을 지배 — 상보성 확인
- 부수 효과: 새 best known 2건 발견 (multi/paper 35,777, single/p36 49,621)

단계 2: 동일 예산(100초) 통합 비교 — compact vs CG

compact 모형은 solver에 100초, CG 변형은 **CG 시간 + RMH 시간 = 100초**로 예산을 통일해 최종 해 품질을 비교 (RMH = 생성된 column만으로 integer master를 푸는 heuristic).

인스턴스	compact 최선 (모형)	CG 최선 (전략)	우위
소형 (paper, p25)	최적 또는 근접	동급	동등
single/p49	64,030 (network)	63,730 (mach_best_k3)	CG
multi/p36	59,125 (network)	58,858 (mach_best_k5)	CG
multi/p49	feasible 실패 [†]	76,712 (mach_best_k3)	CG

[†] base/base_vi/network 모두 100초 내 feasible 해 없음. 당시 기준값이며 이후 단계 5–6에서 추가 갱신됨.

- **규모가 커지면 같은 예산에서 CG의 해가 compact 모형보다 좋다** — pattern+CG의 효용이 bound(network와 동일)가 아니라 해 품질에서 먼저 나타남
- 단, 대형에서 single/loc_best 전략은 CG 수렴에만 수백 초가 들어 예산 초과 — mach_best 계열이 필수

단계 3: flow cover inequality — 효과 없음

아이디어: (operation, 기간)별 "capacity + 수요" 구조는 고정비 유량 문제(single-node fixed-charge flow)와 같다. 이 구조의 표준 cut인 flow cover — 용량 합이 수요를 넘는 위치 집합 $C(\text{cover})$ 에 대해

$$\sum_{p \in C} v_{plt} + (u - \lambda) \sum_{p \in C} (1 - y_p) \leq d_l^t \quad (\lambda = \text{cover 용량} - d_l^t, y_p = \text{위치 } p \text{ 가동 여부})$$

를 cutting plane으로 분리·추가했다. 위치 집계형과 (위치, config) 쌍 수준 2종 구현.

결과: 8개 인스턴스에서 개선 0~0.6%p — **사실상 효과 없음**.

원인 진단: LP의 잔여 분수성은 "느린 config를 분수로 섞어 용량을 딱 맞추는" 형태인데, flow cover는 위치의 가동 여부(y_p)만 보고 어떤 config인지(처리속도 이질성)를 구분하지 못한다. 쌍 수준 변형도 원 변수로 투영하는 과정에서 같은 정보가 소실됨.

원인 분석의 결론

cut family의 효과는 LP relaxation이 허용하는 분수해의 형태와 cut의 계수 구조가 일치하는지에 달려 있다. 단계 4의 θ /MIR cut은 처리속도를 계수에 직접 반영하는 family다.

단계 4: θ -cut과 MIR 강화

valid row $\sum_{p,j} B_{jl} s_{pjlt} \geq d_l^t$ (전체 처리능력 $\geq$ 수요)를 임의의 divisor θ 로 나눠 정수 반올림:

- **θ -cut** (Chvátal–Gomory rounding): $\sum_{p,j} \lceil B_{jl}/\theta \rceil s_{pjlt} \geq \lceil d_l^t/\theta \rceil$, θ = 해당 operation의 서로 다른 처리속도 전부. $\theta = B^{\max}$ 가 기존 counting cut이므로 **이 family가 counting cut을 포함·지배**
- **MIR-cut**: 같은 row에 mixed-integer rounding을 적용해 계수를 $\lfloor B/\theta \rfloor + \min(1, \text{frac}(B/\theta)/f)$ 로 낮춘 강화판 ($f = \text{frac}(d/\theta)$) — θ -cut을 다시 지배
- 정적 추가 (인스턴스당 수십~150개), separation 불필요. validity는 MIP 최적값 보존으로 검증 (22,910 재현)

결과 (network 위에 없었을 때 pure LP gap; 8개 중 4개 발체):

인스턴스	network	+flow cover	+counting cut	+ θ	+MIR
single/paper	3.26%	3.10%	1.30%	1.27%	1.03%
single/p49	6.43%	6.43%	3.74%	3.08%	1.77%
multi/p36	8.00%	8.00%	4.88%	3.85%	3.62%
multi/p49	11.59%	11.39%	8.48%	6.81%	6.32%

전 인스턴스에서 순위 동일: **MIR** > θ > **counting cut** $\gg$ **flow cover**. MIR 위에 flow cover를 추가로 없어도 이득 없음 $\rightarrow$ MIR 단독을 표준 강화로 채택.

단계 5: network+MIR의 MIP 성능 — 최적 증명 2건

network+MIR을 solver에 직접 투입 (100초). 셀 = 얻은 해 / 잔여 gap:

인스턴스	network_vi (단계 1)	network+MIR	best known 갱신
single/paper	22,910 / 0% (1.9초)	22,910 / 0% (1.2초)	— (최적 기존 증명)
single/p25	33,971 / 1.71%	33,915 / 0.73%	33,915 (신규)
single/p36	49,621 / 0.72%	49,621 / 0.31%	—
single/p49	64,114 / 5.33%	63,456 / 2.41%	63,456 (신규)
multi/paper	35,777 / 1.57%	35,791 / 1.64%	35,775 (θ 판이 발견)
multi/p25	35,155 / 1.92%	35,155 / 0% (36초)	최적 증명 (최초)
multi/p36	59,489 / 7.13%	58,777 / 4.67%	58,768 (θ 판이 발견)
multi/p49	161,405 / —	86,819 / 19.1%	(CG가 더 좋음, 단계 6)

- scaled 인스턴스 최초의 최적값 확정: multi/p25 = 35,155 (36초 증명)
- 소형·중형 인스턴스의 잔여 gap은 0~2.4%. 대형(p49급)은 gap이 크게 남음 (단계 6에서 CG와 비교)

단계 6: MIR row를 CG master에 통합

MIR cut은 pattern 변수로도 표현된다 (pattern이 기간별 (config, op)을 알므로 계수 계산 가능) → RMP에 행으로 추가. **pricing 구조는 그대로**: row의 dual이 시간전개 그래프의 노드 비용으로 흡수되어 shortest path 유지 (shared resource 제약과 동일한 메커니즘).

검증: CG 수렴값 = network+cut의 pure LP bound, 16/16 케이스 일치 (DW 관계가 cut 추가 후에도 유지됨을 수치로 확인).

결과 (CG+MIR, mach_best_k3, 총예산 100초의 RMH 해):

인스턴스	network+MIR MIP (단계 5)	CG+MIR RMH	비고
single/p49	63,456	62,859	신규 best known
multi/p36	58,777	57,830	신규 best known
multi/p49	86,819	74,918	신규 best known
소형 5개	최적~근접	동급 이하	compact가 충분

대형에서는 cut을 강화해도 **CG의 해 품질 우위가 유지된다** — "소형·중형 = compact(network+MIR), 대형 = CG" 구도가 일관됨.

단계	내용	결과
1	counting cut × network 결합	LP gap 8/8 지배, 상보성 확인
2	동일 예산 통합 비교	대형에서 CG 해 품질 우위
3	flow cover 2종	효과 없음 — 원인: rate 이질성 소실
4	θ -cut / MIR	MIR이 전부 지배, 표준 강화로 채택
5	network+MIR MIP	최적 증명 2건, 소형·중형 잔여 gap 0~2.4%
6	MIR을 CG master에 통합	검증 16/16, 대형 best known 3건

시사점:

1. 최강 compact 모형 = **network + MIR-cut** (LP gap 1.0~6.3%; base는 8.4~18.0%)
2. 대형 인스턴스의 해 공급원은 **CG+MIR의 RMH** — bound는 compact와 동일(DW), 해는 우위
3. 다음 단계 **branch-and-price**가 이 둘의 결합점: MIR row를 가진 master로 시작하면 root bound가 network+MIR과 같고, CG의 해 품질과 분기 탐색의 최적성 보장을 함께 얻는다

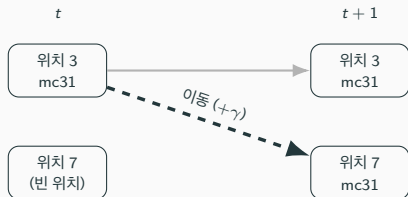
이 절의 best known: 22,910* / 33,915 / 49,621 / 62,859 (single), 35,775 / 35,155* / 57,830 / 74,918 (multi); * = 최적 증명. 앞 절 (§5)의 수치는 7/12 이전 스냅샷.

7. 확장

확장 1: relocation

기준 논문의 “설치 후 위치 고정” 가정을 완화: 기간 경계에서 이동 비용 γ 를 지불하면 기계를 다른 위치로 이동 가능.

모형별 영향: network 모형에서는 위치 간 이동 arc 한 종류를 추가하면 된다.



base 모형 확장: 6-index 전이변수가 필요하고, 동일 기계가
유지와 이동의 source로 **중복 사용되는 해**를 막으려면
위치·configuration별 outflow 등식이 추가로 필요
(binary 62,000개 vs network 1,600개, Example 2 기준)

결과 — relocation의 가치는 수요 구조에 의존:

- Example 1: 이동 비용을 0으로 두어도 절감이 16 (0.07%)임을 증명. 수요가 비율만 증가하는 구조여서 초기 배치가 전 기간 최적
- Example 2: 부품 구성이 기간별로 변함(A 감소, C 증가) → 이동이 최적해에 포함되고 목적함수 개선 ($35,763 < 35,791$). 즉 **수요 패턴이 변하는 경우에만 relocation이 유효**

확장 2: shared resource

auxiliary module의 보유량이 유한한 경우: “동일 기간에 module u 를 장착한 기계 수 $\leq U_u$ ” 제약을 추가.

모형별 영향: 세 모형 모두 (module, 기간)별 선형 제약 1개 추가로 충분 (새 변수·목적함수 변경 없음).

결과 (Example 1, 최다 사용 module(12대 동시 사용)에 한도 부과):

시나리오	최적값	base	network	pattern
한도 없음	22,910	3개 모형 최적값 일치 (검증)		
한도 11대	23,085 (+175)	100초 내 미증명	23초	57초
한도 10대	23,260 (+350)	100초 내 미증명	38초	59초

구현 과정에서 확인된 주의점 (pattern 모형)

(operation, 처리속도) profile 기반 dominance pruning은 shared resource 제약 하에서 부정확한 모형이 된다
— module 구성이 다른 pattern은 상호 대체 불가. 실제로 최적값 +81인 해가 “최적”으로 증명되는 오류를 확인, profile에 module 구성을 포함해 수정. 동일 원칙이 column generation 설계에도 적용된다.

8. 요약

1. base model의 LP bound가 약한 원인을 규명 (fractional 구매, 재구성 비용 과소계상). 부수적으로 기존 논문의 reported 최적값 2건이 suboptimal임을 확인.
2. machine lifecycle을 경로로 정식화하는 **network reformulation**이 동일 최적해를 4.5배 빠르게 도출하고, 규모가 클수록 우위가 확대 (최대 인스턴스에서 잔여 gap 25.1% vs 6.7%). 확장(relocation, shared resource)도 이 구조에서 자연스럽게 수용됨.
3. pattern 모형은 network와 LP bound가 동일(DW 관계)하며, 변수 열거가 한계에 도달 → **column generation**이 다음 단계.

향후 계획 / 역할 분담 (7/13 갱신 — CG·VI 강화는 §6에서 완료됨):

- 공통: 코드 재현 (전 실험이 스크립트로 재실행 가능, 결과는 JSON으로 저장됨)
- A: network+MIR 모형 운용 — 대형 인스턴스 장시간 run으로 최적값 확정, relocation의 multi-part 실험
- B: **branch-and-price 구현** (colgen.py 기반; MIR row를 가진 master, divergence node branching)
- C: MIR family의 이론 정리(dominance 증명), 인스턴스 생성 캠페인 자동화 (크기 $\times$ seed 그리드)

자료: docs/model_formulation.tex(수식), docs/research_directions.md(전체 결과), implementation/(코드·실험 스크립트).

[백업] LP bound 진단: 3단계 분해

예제	기준값	pure LP (gap)	+MIP presolve (gap)	+Gurobi cuts (gap)
Example 1	22,910	20,925 (8.66%)	20,950 (8.56%)	22,137 (3.37%)
Example 2	35,791	32,776 (8.42%)	32,915 (8.03%)	33,979 (5.06%)

presolve의 기여는 0.1–0.4%p로 미미. general cut이 gap의 약 절반을 회복하나 3.4–5.6%가 잔존 → 문제 구조를 반영한 reformulation의 근거.

[백업] relocation: base 스타일 vs network 스타일 인코딩

γ	인코딩	Ex1: obj / gap / time	Ex2: obj / gap / 이동 횟수
10^9	base형 (BR)	22,910 / 0% / 10.6s	35,815 / 2.48% / 0
10^9	network형 (NR)	22,910 / 0% / 28.8s	36,507 / 5.57% / 0
20	BR	22,910 / 0% / 77.3s	36,008 / 4.76% / 3
20	NR	22,910 / 0.83% / limit	36,031 / 4.43% / 5
0	BR	22,894 / 0.79% / limit	35,823 / 4.08% / 50
0	NR	22,894 / 0% / 72.1s	35,763 / 3.90% / 50

두 인코딩은 등가이며 pure LP bound도 전 케이스에서 일치. 모형 크기는 BR이 binary 62,000 / 제약 123,591, NR이 1,600 / 3,708 (Example 2). 이 규모에서는 runtime 우열이 일관되지 않으며, 구조적 차이는 BR의 전이변수가 $|P|^2$ 로 증가한다는 점.

[백업] 규모 확장 상세

인스턴스	base	base+cut	network	pattern	pattern λ 수
p25 (25/6/4)	6.51%	3.80%	3.49%	4.16%	61,700
p36 (36/8/6)	11.52%	5.43%	4.85%	4.73%	144,720
p49 (49/10/8)	25.14%	17.61%	6.72%	7.19%	231,574

network LP bound – base LP bound: +5.9% $\rightarrow$ +8.3% $\rightarrow$ +14.1% (규모에 따라 확대). best known incumbents: 33,947 / 49,734 / 63,920 (모두 최적성 미증명 — 장시간 run으로 확정 예정).